

Generative AI Text Detection Engine

Complete Technical Documentation & Architectural Reference Manual

Document Version: 2.0 (High-Performance)	Date: September 21, 2026
Primary Classifier: Fine-Tuned RoBERTa Transformer	Perplexity Engine: DistilGPT2 Causal LM
Synthesis Engine: LangChain + Gemini 3.6 Flash	Database & UI: SQLite + Streamlit Lab UI

EXECUTIVE SUMMARY: The Voight-Kampff AI Detection System is an end-to-end NLP framework designed to accurately identify AI-generated text versus human-authored content. Combining deep transformer classification (RoBERTa) with linguistic metrics (lexical diversity, burstiness) and causal language model perplexity (DistilGPT2), the platform delivers robust detection scores alongside GenAI-synthesized explanations.

1. System Overview & Objectives

As Large Language Models (LLMs) proliferate, distinguishing synthetic machine text from organic human prose has become critical for academic integrity, media authenticity, and cybersecurity. The Voight-Kampff project provides a reliable, multi-signal forensic analysis tool inspired by dual-processing NLP methodology.

Key Objectives:

- High-Accuracy Classification:** Fine-tuned RoBERTa transformer model serving as the primary detection authority.
- Multi-Signal Verification:** Supporting statistical signals including Lexical Diversity, Sentence Burstiness, and DistilGPT2 Perplexity.
- High-Speed Inference:** Caching mechanisms (`@st.cache\_resource`, `lru\_cache`) delivering analysis results in sub-second time (0.16s).
- Explainable AI (XAI):** Automated plain-English explanations via Gemini 3.6 Flash via LangChain.
- Persistence & Auditability:** SQLite storage and automated PDF report compilation.

2. System Architecture & Components

The architecture is structured in a modular fashion, separating data preprocessing, model inference, metric evaluation, database logging, and front-end visualization.

Component Module	Target Path	Core Responsibility
Master Entrypoint	train.py	Orchestrates dataset preprocessing and transformer training.
Data Preprocessor	data/prepare_dataset.py	Auto-detects columns, handles numeric/string labels, balances data.
Transformer Classifier	transformer/predict.py	RoBERTa sequence classifier with singleton lazy model caching.
Perplexity Engine	nlp/perplexity.py	DistilGPT2 cross-entropy loss computation for predictability.
Burstiness & NLP	nlp/burstiness.py nlp/linguistic_features.py	Sentence length variance, lexical diversity (TTR), word counts.
GenAI Explanation	genai/explanation.py	LangChain prompt pipeline with Gemini for human-readable reports.
Database Engine	database/database.py	SQLite database creation and historical record tracking.
PDF Report Generator	reports/report_generator.py	ReportLab PDF generation for individual analysis audits.
Web Application	app.py	Streamlit interactive lab dashboard with tabbed UI and dark theme.

3. Data Preparation & Model Training Pipeline

Dataset Auto-Detection & Cleaning

The dataset preprocessor (data/prepare_dataset.py) handles large datasets (e.g. 1.11 GB / 487,235 raw rows) without memory overhead. It features intelligent auto-detection for text and label columns across diverse dataset formats:

- **Text Column Matchers:** ['text', 'text_content', 'content', 'document', 'body', 'essay']
- **Label Column Matchers:** ['generated', 'label', 'target', 'class', 'is_ai']
- **Label Normalization:** Automatically converts numeric floats (0.0 / 1.0) and strings ('human' / 'ai') into binary integers (0 = Human, 1 = AI).

- **Stratified Subsampling:** Extracts an equal distribution (5,000 Human, 5,000 AI) into 80% Train (8,000), 10% Validation (1,000), and 10% Test (1,000) splits.

RoBERTa Fine-Tuning Setup

The primary detector is fine-tuned using PyTorch and HuggingFace Transformers (transformer/train.py):

Hyperparameter	Value / Setting	Rationale
Base Architecture	roberta-base	Robust pre-trained bidirectional transformer encoder.
Max Sequence Length	128 tokens	Optimal trade-off between semantic context & CPU speed.
Optimizer	AdamW (lr=2e-5)	Standard decoupled weight decay optimizer for NLP fine-tuning.
Batch Size	8 (CPU) / 16 (GPU)	Balanced memory consumption and gradient stability.
Epochs	2 Epochs	Prevents overfitting while achieving high validation accuracy.
Scheduler	Linear Warmup (10%)	Smooth learning rate warm-up and decay schedule.

4. Forensic Metrics & Detection Signals

Primary Signal: RoBERTa Classification

The fine-tuned model outputs classification logits for binary classes. Applying a Softmax function yields probability distributions for Human versus AI origin:

```
P(AI) = softmax(Logits)[1], P(Human) = softmax(Logits)[0]
```

Secondary Signal: Perplexity (DistilGPT2)

Perplexity measures how surprised a causal language model is by the text. Synthetic texts generated by LLMs typically exhibit lower perplexity (predictable token sequences), whereas human text demonstrates higher perplexity.

```
Perplexity = exp( CrossEntropyLoss(Text, DistilGPT2) )
```

Secondary Signal: Burstiness Index

Burstiness quantifies the variation in sentence length across a document. Human writers naturally mix concise sentences with long, descriptive structures (high standard deviation). AI generators tend to maintain uniform sentence lengths.

```
Burstiness Score = StdDev(Sentence Lengths) / Mean(Sentence Lengths)
```

Linguistic Feature Suite

Extracts key stylistic markers:

- **Lexical Diversity (Type-Token Ratio):** Ratio of unique words to total words.
- **Average Word Length:** Character count divided by word count.
- **Punctuation Ratio:** Punctuation density relative to total character length.

5. Performance & Speed Optimizations

Prior to optimization, initializing HuggingFace transformer models on every analysis request created a 70-second latency bottleneck. By implementing singleton lazy model loading and Streamlit resource caching (@st.cache_resource / @st.cache_data), the runtime latency was dramatically reduced.

Metric / Stage	Uncached Execution	Optimized Cached Execution	Speedup Factor
Model Cold Load	~70.0 seconds	5.23 seconds (Startup Only)	13.3x Faster
Repeated Text Analysis	~70.0 seconds	0.166 seconds	> 420x Faster
Memory Overhead	Repeated Allocations	Single Resident Process	Stable Footprint

6. User Interface & Operation Manual

Command Line Operations

1. Prepare Dataset & Fine-Tune Model:

```
python train.py
```

2. Launch Web Dashboard:

```
streamlit run app.py
```

Streamlit Dashboard Features

- **Preset Sample Loader:** One-click loading of AI-generated vs Human-written sample texts in the sidebar.
- **Tab 1 - Detection Overview:** Color-coded banner (AI Red vs Human Green), dual progress bars, quick signal summary.
- **Tab 2 - Linguistic Features:** Metric cards for Lexical Diversity, Word Count, Sentence Length, and raw JSON export.
- **Tab 3 - Perplexity & Burstiness:** Detailed breakdown of DistilGPT2 predictability and sentence length variance.
- **Tab 4 - GenAI Synthesis Report:** Gemini 3.6 Flash structured reasoning and confidence assessment.

- **Tab 5 - Audit PDF & SQLite History:** Generate and download downloadable PDF report audits and explore historical scans.

End of Documentation — Voight-Kampff AI Detection Engine v2.0